

1 Manajemen File & User/Group

Bab ini merangkum fondasi administrasi akses di Linux: permission Unix klasik, ACL untuk kasus yang lebih granular, manajemen user dan group, penggunaan `sudo` dan `su`, serta pembatasan ruang disk dengan quota. Fokus bab ini adalah memahami *model kontrol akses* Linux secara utuh: siapa memiliki file, siapa boleh mengakses, bagaimana hak administratif didelegasikan, dan bagaimana penggunaan sumber daya dibatasi di sistem multi-pengguna.

Tujuan Praktikum

Setelah menyelesaikan bab ini, pembaca diharapkan mampu:

1. membaca dan mengubah permission file/direktori;
2. membedakan peran owner, group, dan others;
3. menggunakan ACL saat permission standar tidak cukup;
4. membuat, memodifikasi, dan menghapus user serta group;
5. mengelola hak administratif dengan `sudo` secara aman; dan
6. mengaktifkan serta memeriksa disk quota untuk user atau group.

1.1 Sistem Kontrol Akses (Permissions)

Penjelasan Konsep

Linux mewarisi model kontrol akses Unix yang sederhana namun kuat. Setiap file dan direktori selalu memiliki tiga identitas akses:

Owner pengguna yang memiliki file.

Group group yang terkait dengan file.

Others semua pengguna lain di luar dua entitas di atas.

Untuk masing-masing entitas itu tersedia tiga izin dasar: baca, tulis, dan eksekusi.

Tabel 1.1: Makna izin dasar Linux

Simbol	Pada file	Pada direktori
r	Membaca isi file	Melihat isi direktori
w	Mengubah isi file	Membuat atau menghapus entri di direktori
x	Menjalankan file	Masuk ke direktori

Catatan

Hak w pada file hanya mengubah isi file. Menghapus file ditentukan oleh izin pada direktori yang menampung file tersebut.

Perintah yang Digunakan

- `ls -l`: melihat tipe file, permission, owner, dan group.
- `chmod`: mengubah permission file atau direktori.
- `chown`: mengubah owner dan/atau group pemilik file.
- `chgrp`: mengubah group pemilik file.
- `umask`: melihat atau mengubah permission default file baru.

1.1.1 Membaca Output `ls -l`

Output `ls -l` merangkum tipe file, permission, owner, group, ukuran, dan waktu.

```
$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2847 Jan 15 10:30 /etc/passwd
```

Kode 1.1: Membaca permission dengan `ls -l`

1.1 Sistem Kontrol Akses (Permissions)

Interpretasi string permission:

- karakter pertama: tipe file (-, d, l, dll.)
- tiga karakter berikutnya: permission owner
- tiga berikutnya: permission group
- tiga terakhir: permission others

Contoh `-rwxr-x-` berarti file biasa, owner punya hak penuh, group dapat baca dan eksekusi, others tidak punya akses.

1.1.2 Notasi Simbolik dan Oktal

Permission dapat ditulis dalam dua bentuk:

- **simbolik**: relatif, misalnya `u+x`, `go-w`
- **oktal**: absolut, misalnya `755`, `640`, `600`

Tabel 1.2: Konversi permission ke notasi oktal

Izin	Biner	Oktal	Makna
-	000	0	Tidak ada izin
r-	100	4	Read saja
rw-	110	6	Read dan write
rwx	111	7	Read, write, execute

Pola yang paling umum:

- `755`: direktori atau program publik
- `644`: file biasa
- `600`: file privat seperti kunci SSH
- `700`: skrip atau direktori privat

1.1.3 `chmod`, `chown`, dan `chgrp`

Tiga perintah inti pengelolaan permission adalah:

- `chmod`: mengubah permission
- `chown`: mengubah owner dan/atau group
- `chgrp`: mengubah group saja

```
# chmod simbolik
chmod +x script.sh
chmod u=rw,g=r,o= file.txt
chmod go-w file.txt

# chmod oktal
chmod 644 /etc/app.conf
chmod 755 /usr/local/bin/my-script.sh
chmod 600 ~/.ssh/id_rsa

# ownership
sudo chown alice report.txt
sudo chown alice:developers report.txt
sudo chgrp developers report.txt
```

Kode 1.2: Perintah dasar permission dan ownership

Prinsip pemakaian:

- gunakan notasi simbolik untuk perubahan kecil dan bertahap;
- gunakan notasi oktal saat ingin hasil akhir pasti, terutama di skrip;
- selalu verifikasi perubahan rekursif dengan `ls -la` pada beberapa file.

Peringatan

Hindari `chmod 777` kecuali benar-benar memahami risikonya. Permission ini memberikan hak tulis dan eksekusi ke semua pengguna.

1.1.4 Special Permission Bits

Selain sembilan bit `rwx`, Linux memiliki tiga bit khusus:

SUID program berjalan dengan UID pemilik file. Contoh klasik: `/usr/bin/passwd` perlu menulis ke `/etc/shadow`.

SGID pada file executable proses berjalan dengan GID file; pada direktori, file baru mewarisi group direktori.

Sticky bit pada direktori, pengguna hanya boleh menghapus file miliknya sendiri walau direktori dapat ditulis banyak orang.

```
# SUID
ls -l /usr/bin/passwd
sudo chmod 4755 /usr/local/bin/my-tool

# SGID pada direktori bersama
sudo mkdir /srv/project
sudo chown root:developers /srv/project
sudo chmod 2775 /srv/project

# Sticky bit
ls -ld /tmp
sudo chmod 1777 /shared/uploads
```

Kode 1.3: Contoh special bits

Tips

Kombinasi SGID + sticky bit sering dipakai untuk direktori kolaborasi: SGID menjaga group tetap konsisten, sticky bit mencegah saling hapus file.

1.1.5 umask — Nilai Default Izin Baru

umask adalah masker yang *mengurangi* permission default file baru. Secara praktis, file reguler berangkat dari 666, direktori dari 777, lalu dikurangi oleh nilai umask.

Tabel 1.3: Pengaruh umask terhadap file dan direktori baru

umask	File baru			
	Oktal	Simbolik	Oktal	Simbolik
022	644	rw-r-r-	755	rwxr-xr-x
027	640	rw-r--	750	rwxr-x-
077	600	rw---	700	rwx---

```
umask
umask -S
umask 027
echo "umask 027" >> ~/.bashrc
```

Kode 1.4: Melihat dan mengubah umask

Catatan

umask tidak pernah menambahkan execute pada file reguler. Jika file harus executable, gunakan `chmod` secara eksplisit.

Praktikum 9.1 — Permissions

Tujuan: membaca permission, mengubahnya dengan aman, dan memahami efek `chmod`, `chown`, serta `umask` pada file nyata.

Langkah 1: Buat direktori kerja dan dua file uji.

```
mkdir ~/lab-permissions && cd ~/lab-permissions
echo "data rahasia" > secret.txt
echo '#!/bin/bash' > myscript.sh
echo 'echo Hello' >> myscript.sh
ls -la
```

Kode 1.5: Menyiapkan file uji permission

Perhatikan permission awal kedua file. Biasanya file baru tidak memiliki bit execute.

Langkah 2: Jadikan `secret.txt` privat hanya untuk owner.

```
chmod 600 secret.txt
ls -l secret.txt
```

Kode 1.6: Membuat file privat

Permission `600` berarti owner dapat membaca dan menulis, sedangkan group dan others tidak memiliki akses.

Langkah 3: Jadikan `myscript.sh` dapat dijalankan.

```
chmod 755 myscript.sh
ls -l myscript.sh
./myscript.sh
```

Kode 1.7: Menjadikan skrip executable

Permission `755` memberi hak penuh ke owner, dan hak baca+execute ke group serta others. Tanpa bit execute, file skrip tidak bisa dijalankan langsung.

Langkah 4: Buat direktori bersama dan amati efek SGID sederhana.

```
mkdir shared-dir
chmod g+s shared-dir
ls -ld shared-dir
```

Kode 1.8: Direktori bersama dengan SGID

Jika output menampilkan huruf `s` pada posisi group execute, berarti SGID aktif.

Langkah 5: Uji efek `umask` pada file baru.

```
umask
umask 027
touch testfile-027
ls -l testfile-027
```

Kode 1.9: Mengamati efek umask

Analisis

1. Mengapa `secret.txt` tidak dapat dibaca oleh group dan others setelah `chmod 600`?
2. Apa perbedaan arti `600` dan `755` terhadap file yang diuji?
3. Setelah `umask 027`, permission apa yang dihasilkan untuk file baru, dan mengapa bukan `777`?

Tantangan

Ubah owner atau group salah satu file uji ke akun atau group lain yang tersedia di sistem, kemudian jelaskan perubahan output `ls -l` sebelum dan sesudahnya.

1.2 Access Control Lists (ACLs)

Permission Unix klasik sangat cocok untuk mayoritas kasus, tetapi ia hanya mengenal satu owner dan satu group. Masalah muncul saat akses perlu diberikan ke satu pengguna tertentu tanpa mengubah owner file dan tanpa memasukkan pengguna itu ke group utama.

Contoh masalah yang umum:

Contoh: Mengapa ACL diperlukan

File laporan.txt dimiliki alice dan group finance. User bob perlu membaca file itu, tetapi Anda tidak ingin semua anggota group lain ikut mendapat akses. ACL memungkinkan Anda memberi izin hanya ke bob.

ACL menambahkan aturan akses *tambahan* di atas permission standar. Ada dua jenis:

Access ACL berlaku langsung pada file atau direktori saat ini.

Default ACL hanya untuk direktori, dan diwariskan ke konten baru di dalamnya.

Catatan

Di Ubuntu 22.04, filesystem modern seperti ext4, XFS, dan Btrfs umumnya sudah mendukung ACL. Jika utilitasnya belum tersedia, instal paket `acl`.

Perintah yang Digunakan

- `getfacl file`: menampilkan ACL lengkap file atau direktori.
- `setfacl -m ...`: menambah atau mengubah entri ACL.
- `setfacl -x ...`: menghapus satu entri ACL.
- `setfacl -b file`: menghapus semua ACL tambahan.
- `setfacl -d -m ... dir`: menetapkan *default ACL* pada direktori.

1.2.1 Langkah 1: Membaca ACL yang Ada

Perintah pertama yang perlu dikenal adalah `getfacl`. Fungsinya mirip `ls -l`, tetapi lebih detail.

```
sudo apt install -y acl
getfacl file.txt

# contoh output minimum
# user::rw-
# group::r--
# other::r--
```

Kode 1.10: Membaca ACL dasar dengan `getfacl`

Jika file belum memiliki ACL tambahan, output `getfacl` hanya menampilkan tiga entri dasar yang sepadan dengan permission Unix biasa: owner, group, dan others. Begitu ACL tambahan diterapkan, Anda akan melihat entri user/group spesifik dan sering juga sebuah mask.

Format entri yang paling penting adalah:

Tabel 1.4: Format entri ACL yang umum

Entri	Contoh	Arti
u::izin	u::rw-	Permission owner
u:nama:izin	u:bob:r-	ACL untuk user tertentu
g::izin	g::r-	Permission group standar
g:nama:izin	g:audit:rw-	ACL untuk group tertentu
o::izin	o::-	Permission others
m::izin	m::rw-	Mask ACL

Ketika ACL aktif, `ls -l` menampilkan tanda + di akhir string permission, misalnya `-rw-r--+`.

1.2.2 Langkah 2: Menambahkan ACL untuk Satu User atau Group

Perintah utama untuk membuat atau mengubah ACL adalah `setfacl`. Mulailah dari kasus paling sederhana: memberi akses baca ke satu user.

```
# beri bob hak baca
setfacl -m u:bob:r file.txt

# beri group audit hak baca-tulis
setfacl -m g:audit:rw file.txt

# lihat hasilnya
getfacl file.txt
```

Kode 1.11: Menambahkan ACL secara bertahap

Cara membacanya:

- `-m` berarti *modify*: tambah atau ubah entri;
- `u:bob:r` berarti user bob mendapat izin baca;
- `g:audit:rw` berarti group audit mendapat izin baca dan tulis.

Jika ACL tidak lagi diperlukan, hapus salah satu entri atau semua entri tambahan:

```
setfacl -x u:bob file.txt
setfacl -x g:audit file.txt
setfacl -b file.txt
```

Kode 1.12: Menghapus ACL

`-x` menghapus satu entri tertentu. Opsi `-b` menghapus seluruh ACL tambahan dan mengembalikan file ke model permission Unix standar.

1.2.3 Langkah 3: Memahami Mask dan Izin Efektif

Begitu ACL tambahan dipasang, Linux biasanya membuat **mask**. Untuk pemula, bayangkan mask sebagai *batas atas* izin yang boleh benar-benar berlaku untuk entri ACL tertentu.

```
getfacl ~/report.txt
# user::rw-
# user:charlie:rw-          #effective:rw-
# group::r--                #effective:r--
# group:audit:rw-           #effective:rw-
# mask::rw-
# other::---

setfacl -m m::r-- ~/report.txt
chmod g-w ~/report.txt
```

Kode 1.13: Mask ACL dan izin efektif

Pada contoh di atas, `user:charlie:rw-` tidak benar-benar mendapat `rw-` karena mask hanya mengizinkan `rw-`. Itulah sebabnya `getfacl` menampilkan komentar `#effective:rw-`.

Aturan praktis yang perlu diingat:

- owner dan others tidak difilter oleh mask;
- named user, group standar, dan named group bisa dipersempit oleh mask;
- `chmod` pada file ber-ACL sering memengaruhi mask;
- jika hasilnya membingungkan, selalu cek lagi dengan `getfacl`.

1.2.4 Langkah 4: Default ACL untuk Direktori

Setelah paham ACL pada satu file, langkah berikutnya adalah ACL yang diwariskan otomatis. Inilah fungsi **default ACL**. Ia sangat berguna untuk direktori proyek bersama.

```
mkdir ~/project
setfacl -d -m g:developers:rwX ~/project
setfacl -d -m u:deploy:r-x,g:auditors:r-- ~/project

touch ~/project/newfile.txt
getfacl ~/project/newfile.txt

mkdir ~/project/subdir
getfacl ~/project/subdir

setfacl -x d:g:developers ~/project
setfacl -k ~/project
```

Kode 1.14: Default ACL pada direktori kolaborasi

Alur kerjanya bertahap:

1. tambahkan default ACL ke direktori dengan opsi `-d -m`;
2. buat file atau subdirektori baru;
3. periksa ACL hasil warisan dengan `getfacl`;
4. hapus default ACL bila tidak lagi dibutuhkan.

Catatan

File baru tidak mewarisi execute dari default ACL bila ia file reguler; subdirektori tetap mempertahankan execute karena `x` pada direktori berarti izin masuk.

Praktikum 9.2 — ACL

Tujuan: memahami ACL dari nol: melihat kondisi awal, menambah akses untuk satu user, lalu membuat direktori yang mewariskan ACL otomatis.

Langkah 1: Siapkan file dan lihat permission standar tanpa ACL tambahan.

```
mkdir ~/lab-acl && cd ~/lab-acl
echo "Data penting" > confidential.txt
chmod 640 confidential.txt
ls -l confidential.txt
getfacl confidential.txt
```

Kode 1.15: Menyiapkan file uji ACL

Pada tahap ini, `getfacl` hanya menampilkan tiga entri dasar: owner, group, dan others. Belum ada named user atau named group.

Langkah 2: Beri akses baca ke satu user tertentu tanpa mengubah owner atau group.

```
setfacl -m u:userA:r confidential.txt
ls -l confidential.txt
getfacl confidential.txt
```

Kode 1.16: Menambahkan ACL untuk satu user

Perhatikan dua perubahan:

- output `ls -l` menampilkan tanda `+`;
- output `getfacl` kini memiliki entri `user:userA:r-`.

Langkah 3: Buat direktori bersama yang mewariskan ACL ke file baru.

```
mkdir shared
setfacl -d -m u:userA:rwX shared
setfacl -d -m u:userB:r-x shared
getfacl shared

touch shared/inherited.txt
getfacl shared/inherited.txt
```

Kode 1.17: Default ACL pada direktori

Analisis

1. Mengapa `getfacl confidential.txt` awalnya tidak menampilkan user tertentu?
2. Setelah `setfacl -m u:userA:r confidential.txt`, apa perbedaan output `ls -l` dan `getfacl`?
3. Mengapa file `inherited.txt` mewarisi ACL dari direktori `shared`?

Tantangan

Tambahkan satu ACL lagi agar group `readonly-group` hanya dapat membaca `confidential.txt`. Setelah itu, hapus ACL untuk `userA` dan verifikasi hasil akhirnya dengan `getfacl`.

1.3 Manajemen User dan Group

Penjelasan Konsep

Linux menyimpan identitas pengguna di tiga file: `/etc/passwd` (data dasar user, dapat dibaca semua orang), `/etc/shadow` (hash password dan aging policy, hanya root), dan `/etc/group` (definisi group). Sistem ini memisahkan informasi publik dari rahasia keamanan.

Perintah Utama

- `useradd`, `usermod`, `userdel`: kelola akun user.
- `groupadd`, `groupmod`, `groupdel`: kelola group.
- `passwd`: ubah atau kunci password.
- `chage`: atur kebijakan umur password.
- `id`, `groups`: lihat informasi user dan group.

1.3.1 Membaca Informasi User dan Group

```
id alice
groups alice
getent passwd alice
getent group developers
grep "^alice:" /etc/passwd
sudo grep "^alice:" /etc/shadow
```

Kode 1.18: Perintah query user dan group

Catatan

`id` lebih akurat daripada `grep /etc/group` karena menampilkan semua group user, termasuk primary group.

1.3.2 Siklus Hidup User: Buat, Modifikasi, Hapus

```
# buat user dengan home directory dan shell
sudo useradd -m -s /bin/bash alice
sudo passwd alice

# modifikasi: tambah group, ubah shell, lock/unlock
sudo usermod -aG developers,sudo alice
sudo usermod -s /bin/zsh alice
sudo usermod -L alice
sudo usermod -U alice

# hapus user (dengan atau tanpa home directory)
sudo userdel alice
sudo userdel -r alice
```

Kode 1.19: Pengelolaan akun user

Opsi penting:

- `-m`: buat home directory otomatis
- `-s`: tentukan shell login
- `-aG`: tambah group tanpa menghapus group lama

- -L/-U: lock/unlock akun
- -r: hapus home directory saat menghapus user

Peringatan

Gunakan `usermod -aG`, bukan `usermod -G`. Tanpa `-a`, semua supplementary group lama akan diganti.

Praktikum 9.3A — Membuat dan Mengelola User

Tujuan: membuat user baru, memodifikasi propertinya, dan memahami perbedaan opsi `useradd` dan `usermod`.

```
# buat dua user
sudo useradd -m -s /bin/bash userA
sudo useradd -m -s /bin/bash userB
sudo passwd userA
sudo passwd userB

# verifikasi
id userA
getent passwd userA

# modifikasi shell userA
sudo usermod -s /bin/zsh userA
getent passwd userA

# lock dan unlock userB
sudo usermod -L userB
sudo passwd -S userB
sudo usermod -U userB
sudo passwd -S userB
```

Kode 1.20: Latihan user management

Pertanyaan:

1. Apa perbedaan output `id userA` sebelum dan sesudah menambah group?
2. Bagaimana status `passwd -S userB` berubah saat akun di-lock?

1.3.3 Pengelolaan Group

```
sudo groupadd developers
sudo groupadd -g 2000 finance
sudo groupmod -n devteam developers
sudo gpasswd -a alice devteam
sudo gpasswd -d alice devteam
sudo groupdel devteam
```

Kode 1.21: Operasi group

Catatan

Perubahan group baru terasa penuh setelah user logout-login atau membuka subshell dengan `newgrp nama-group`.

Praktikum 9.3B — Group Management

Tujuan: membuat group, menambahkan user ke group, dan memverifikasi keanggotaan.

```
# buat dua group
sudo groupadd labgroup
sudo groupadd readonly-group
```

```
# tambahkan userA ke kedua group
sudo usermod -aG labgroup,readonly-group userA

# tambahkan userB hanya ke readonly-group
sudo usermod -aG readonly-group userB

# verifikasi
id userA
id userB
getent group labgroup
getent group readonly-group
```

Kode 1.22: Latihan group management

Pertanyaan:

1. Apa yang ditampilkan `id userA` vs `groups userA`?
2. Mengapa `-a` pada `usermod -aG` penting?

1.3.4 Kebijakan Password dan Aging

```
# lihat status aging
sudo chage -l alice

# set kebijakan: password valid 90 hari, warning 14 hari
# sebelumnya
sudo chage -M 90 -m 7 -W 14 alice

# kunci akun 30 hari setelah password expired
sudo chage -I 30 alice

# set tanggal kedaluwarsa akun
sudo chage -E 2025-12-31 alice

# paksa ganti password segera
sudo chage -d 0 alice
sudo passwd -e alice
```

Kode 1.23: Password policy dengan chage

Tabel 1.5: Opsi `chage` yang paling penting

Opsi	Nama	Arti
-l	list	Tampilkan status aging
-M N	maxdays	Password wajib diganti tiap N hari
-m N	mindays	Minimal N hari sebelum boleh ganti
-W N	warndays	Peringatan N hari sebelum expired
-I N	inactive	Kunci akun N hari setelah expired
-E DATE	expire	Kedaluwarsa akun

Praktikum 9.3C — Password Aging Policy

Tujuan: menerapkan kebijakan umur password dan mengamati efeknya.

```
# set aging policy untuk userA
sudo chage -M 60 -W 7 -m 1 userA
sudo chage -l userA

# paksa userA ganti password saat login pertama
sudo chage -d 0 userA

# kunci password userB
sudo passwd -l userB
sudo passwd -S userB

# unlock kembali
sudo passwd -u userB
sudo passwd -S userB
```

Kode 1.24: Latihan password policy

Pertanyaan:

1. Apa arti nilai yang ditampilkan `chage -l userA`?
2. Bagaimana cara membuktikan userB terkunci dari output `passwd -S`?
3. Kapan sebaiknya menggunakan `chage -d 0` vs `passwd -e`?

Tantangan

Buat user bernama intern yang:

- memiliki shell `/bin/bash`;
- menjadi anggota labgroup;
- dipaksa ganti password pada login pertama;
- password expired setelah 45 hari dengan warning 7 hari sebelumnya.

1.4 Konfigurasi sudo dan su

Penjelasan Konsep

1.4.1 Perbedaan su dan sudo

`su` berpindah identitas pengguna, sedangkan `sudo` menjalankan perintah tertentu dengan privilege lebih tinggi berdasarkan aturan kebijakan.

Tabel 1.6: Perbandingan `su` dan `sudo`

Aspek	<code>su</code>	<code>sudo</code>
Autentikasi	Password user target	Password pengguna sendiri
Scope	Beralih penuh ke sesi lain	Menjalankan perintah tertentu
Logging	Minimal / tidak default	Tercatat di log sistem
Granularitas	Kasar	Sangat granular
Praktik modern	Lebih terbatas	Direkomendasikan

Catatan

Di Ubuntu, akun root biasanya dinonaktifkan untuk login langsung; karena itu `sudo` menjadi mekanisme standar administrasi.

Perintah yang Digunakan

- `su - user`: berpindah ke user lain dengan login environment penuh.
- `sudo perintah`: jalankan satu perintah dengan privilege lebih tinggi.
- `sudo -i`: buka shell root dengan login environment.
- `sudo -l`: lihat hak sudo yang tersedia.
- `visudo`: edit aturan sudoers dengan validasi sintaks.

1.4.2 Pemakaian Dasar su dan sudo

```
# su
su - alice
su - alice -c "ls -la /home/alice"

# sudo
sudo apt update
sudo -u alice cat /home/alice/report.txt
sudo -i
sudo -s
sudo -l
sudo -k
sudo -e /etc/hosts
```

Kode 1.25: Perintah su dan sudo yang paling sering dipakai

Pedoman ringkas:

- gunakan `su - user`, bukan `su user`, jika memang harus berpindah sesi;
- gunakan `sudo -i` untuk shell root dengan login environment penuh;
- gunakan `sudo -e` atau `sudoedit` untuk mengedit file sistem lebih aman.

1.4.3 Konfigurasi sudoers yang Aman

Konfigurasi utama sudo berada di `/etc/sudoers`, tetapi praktik terbaik adalah menyimpan aturan tambahan di `/etc/sudoers.d/`.

Peringatan

Selalu edit aturan sudo dengan `visudo`. Kesalahan sintaks dapat mengunci akses administratif Anda ke sistem.

Format aturan dasar:

```
WHO
WHERE=(AS_WHO)
COMMAND
```

```
# akses penuh untuk user tertentu
alice ALL=(ALL:ALL) ALL

# akses apt tanpa password
bob ALL=(root) NOPASSWD: /usr/bin/apt

# group sudo bawaan Ubuntu
%sudo ALL=(ALL:ALL) ALL

# izin terbatas untuk service nginx
%webadmin ALL=(root) /bin/systemctl start nginx, \
    /bin/systemctl stop nginx, \
    /bin/systemctl restart nginx, \
    /bin/systemctl status nginx
```

Kode 1.26: Contoh aturan sudoers yang representatif

```
sudo visudo
sudo visudo -f /etc/sudoers.d/developers
sudo visudo -c
sudo grep sudo /var/log/auth.log | tail -20
```

Kode 1.27: Mengelola file sudoers

Praktikum 9.4 — Konfigurasi sudo

Tujuan: membuat aturan sudo terbatas, memverifikasi hak akses, dan membaca log.

Langkah 1: Buat file konfigurasi sudo khusus untuk userA.

```
sudo visudo -f /etc/sudoers.d/lab-userA
```

Kode 1.28: Membuat file sudoers terpisah

Perintah ini membuka editor aman khusus untuk file sudoers baru. Jika sintaks salah, `visudo` akan memperingatkan sebelum file disimpan.

Isi file dengan aturan berikut:

```
userA ALL=(root) NOPASSWD: /usr/bin/apt update, /usr/bin/apt
upgrade
userA ALL=(root) /bin/systemctl status *
```

Kode 1.29: Isi konfigurasi sudo untuk userA

Baris pertama berarti userA boleh menjalankan dua perintah `apt` tanpa password. Baris kedua berarti userA boleh melihat status service apa pun, tetapi tetap mengikuti kebijakan autentikasi normal.

Langkah 2: Verifikasi aturan yang aktif dan uji hasilnya.

```
sudo -l -U userA
sudo grep "userA" /var/log/auth.log | tail -10
```

Kode 1.30: Verifikasi hak sudo dan log

`sudo -l -U userA` dipakai untuk mengecek aturan yang aktif dari sudut pandang akun userA. Log di `/var/log/auth.log` membantu memverifikasi bahwa pemakaian sudo benar-benar tercatat.

Analisis

1. Mengapa aturan disimpan di `/etc/sudoers.d/`, bukan langsung di `/etc/sudoers`?
2. Mana perintah yang bisa dijalankan tanpa password, dan mana yang masih perlu autentikasi?
3. Informasi apa saja yang dicatat di log sudo?

Tantangan

Tambahkan satu aturan baru agar userA boleh menjalankan `/bin/systemctl restart ssh` tetapi tidak boleh menjalankan `reboot`.

1.5 Disk Quota

Penjelasan Konsep

Quota membatasi penggunaan ruang disk dan/atau jumlah file milik user atau group. Fungsinya sederhana: mencegah satu pengguna menghabiskan seluruh sumber daya storage.

1.5.1 Konsep Disk Quota

Ada dua dimensi quota:

Block quota membatasi total ruang disk yang dipakai. Di Linux, tools quota umumnya memakai satuan kilobyte.

Inode quota membatasi jumlah file/direktori, terlepas dari ukurannya.

Selain itu, ada tiga istilah kebijakan yang wajib dipahami:

Catatan

Inode quota penting saat pengguna membuat sangat banyak file kecil. Ruang disk mungkin masih ada, tetapi inode filesystem bisa habis lebih dulu.

Perintah yang Digunakan

- `quotacheck`: membuat atau memperbarui database quota.
- `quotaon/quotaoff`: mengaktifkan atau menonaktifkan enforcement.
- `edquota`: mengedit limit quota secara interaktif.
- `setquota`: menetapkan limit quota dari command line.
- `quota`, `repquota`: melihat penggunaan quota.

Tabel 1.7: Jenis limit pada disk quota

Jenis	Berlaku pada	Makna
Soft limit	Blok dan inode	Boleh dilampaui sementara selama grace period
Hard limit	Blok dan inode	Tidak boleh dilampaui sama sekali
Grace period	Soft limit	Waktu toleransi sebelum soft limit diperlakukan seperti hard limit

1.5.2 Mengaktifkan Quota

Workflow aktivasi quota biasanya terdiri dari empat langkah:

1. instal paket quota,
2. aktifkan opsi `usrquota` dan/atau `grpquota` di filesystem,
3. buat database quota dengan `quotacheck`,
4. aktifkan enforcement dengan `quotaon`.

```
sudo apt install -y quota quotatool

# /etc/fstab
# UUID=abc123 /home ext4 defaults,usrquota,grpquota 0 2

sudo mount -o remount /home
sudo quotacheck -cugm /home
sudo quotaon -v /home
sudo repquota -s /home
```

Kode 1.31: Workflow dasar aktivasi quota

1.5.3 Menetapkan dan Melihat Quota

Perintah yang paling penting adalah:

```
# editor interaktif
sudo edquota -u alice
sudo edquota -g developers

# tanpa editor
sudo setquota -u alice 1048576 1572864 10000 15000 /home

# lihat status
quota -s
sudo quota -u alice
sudo repquota -s /home

# grace period
sudo edquota -t
```

Kode 1.32: Perintah inti pengelolaan quota

Makna argumen pada `setquota -u alice 1048576 1572864 10000 15000 /home`:

- soft block = 1 GB
- hard block = 1,5 GB
- soft inode = 10.000
- hard inode = 15.000

1.5.4 Operasi Lanjutan yang Perlu Dikenal

```
# salin quota dari satu user ke user lain
sudo edquota -p alice bob charlie

# kirim peringatan email
sudo warnquota

# matikan quota
sudo quotaoff -v /home
```

Kode 1.33: Operasi quota lanjutan

Praktikum 9.5 — Disk Quota

Tujuan: memahami alur quota secara aman pada loopback filesystem tanpa mengubah filesystem utama.

Langkah 1: Buat image filesystem kecil dan mount dengan opsi quota.

```
sudo dd if=/dev/zero of=/tmp/quota-test.img bs=1M count=100
sudo mkfs.ext4 /tmp/quota-test.img
sudo mkdir -p /mnt/quota-test
sudo mount -o loop,usrquota,grpquota /tmp/quota-test.img /mnt/
quota-test
```

Kode 1.34: Menyiapkan filesystem uji quota

Image file dipakai agar praktikum aman: Anda tidak perlu memodifikasi filesystem utama seperti /home/. Opsi `usrquota`, `grpquota` mengaktifkan dua jenis quota sekaligus.

Langkah 2: Buat database quota dan aktifkan enforcement.

```
sudo quotacheck -cug /mnt/quota-test
sudo quotaon -v /mnt/quota-test
sudo repquota /mnt/quota-test
```

Kode 1.35: Inisialisasi dan aktivasi quota

`quotacheck -cug` membuat database user dan group quota. Setelah itu, `quotaon` mengaktifkan enforcement, dan `repquota` menampilkan laporan awal.

Langkah 3: Tetapkan quota untuk user uji dan amati hasilnya.

```
sudo edquota -u userA
# contoh: soft block 5120, hard block 10240
sudo repquota /mnt/quota-test
```

Kode 1.36: Menetapkan quota untuk userA

Nilai di atas memakai satuan KB. Jadi 5120 berarti sekitar 5 MB, dan 10240 berarti sekitar 10 MB.

Langkah 4: Bersihkan lingkungan uji setelah selesai.

```
sudo quotaoff /mnt/quota-test
sudo umount /mnt/quota-test
sudo rm /tmp/quota-test.img
```

Kode 1.37: Pembersihan quota test

Analisis

1. Apa perbedaan soft limit dan hard limit saat quota mulai terlampaui?
2. Mengapa praktikum ini memakai loopback filesystem, bukan langsung /home/?
3. Dari output `repquota`, informasi apa yang menunjukkan quota sudah aktif?

Tantangan

Coba atur quota baru untuk userA dengan batas inode yang sangat kecil, kemudian jelaskan kapan pembatasan inode lebih penting daripada pembatasan block.

1.6 Rangkuman

Bab ini membahas lima kelompok konsep inti:

Permissions owner, group, others; izin rwx; `chmod`, `chown`, `chgrp`; special bits; dan `umask`.

ACL perluasan permission Unix untuk kebutuhan akses granular per-user atau per-group, termasuk default ACL direktori dan mask.

User/Group identitas pengguna disimpan di `/etc/passwd`, `/etc/shadow`, dan `/etc/group`; dikelola dengan keluarga perintah `useradd`, `usermod`, `userdel`, `groupadd`, dan `gpasswd`.

sudo dan su `sudo` adalah mekanisme delegasi hak administratif yang lebih aman dan lebih dapat diaudit dibanding `su`.

Disk Quota pembatasan blok dan inode melalui soft limit, hard limit, grace period, serta perintah seperti `edquota`, `setquota`, dan `repquota`.

1.7 Latihan

Latihan Latihan 9.A — Audit dan Kolaborasi

1. Temukan file SUID aktif dengan `find / -perm -4000 -type f 2>/dev/null`, lalu jelaskan tiga file yang Anda kenali beserta alasannya.
2. Cari direktori *world-writable* dan tentukan mana yang valid dan mana yang berisiko.
3. Rancang konfigurasi permission standar dan ACL untuk direktori proyek `/srv/webapp/` agar group `webapp-team` dapat menulis, user `deploy` hanya membaca, dan file baru selalu mewarisi group proyek.

Latihan Latihan 9.B — Kebijakan Akun dan Quota

Tuliskan langkah untuk membuat user intern, menambahkannya ke group `labgroup`, memaksa pergantian password tiap 45 hari (warning 7 hari), memberi izin `sudo` hanya untuk `systemctl status`, dan menetapkan quota ruang serta inode sederhana pada `/home/`.

1.8 Referensi Tambahan

- **Halaman manual:** `man chmod`, `man chown`, `man setfacl`, `man useradd`, `man sudoers`, `man edquota`
- **FHS:** refspecs.linuxfoundation.org/fhs.shtml
- **Ubuntu User Management:** ubuntu.com/server/docs/security-users
- **POSIX ACL:** usenix.org/.../gruenbacher.pdf
- **sudoers(5):** sudo.ws/docs/man/sudoers.man/